

搜索的疆域

搜索与回溯 · 8题 · C++ & Python 双语对照

NQ130 DFS试炼之排列数字 AcWing 842

给定一个整数 n ，将数字 $1\sim n$ 排成一排，将会有很多种排列方法。现在，请你按照字典序将所有的排列方法输出。数据范围： $1\leq n\leq 7$

输入共一行，包含一个整数 n 。**输出**

按字典序输出所有排列方案，每个方案占一行。

来源

AcWing 842

输入

3

输出

```
1 2 3
1 3 2
2 1 3
2 3 1
3 1 2
3 2 1
```

提示 Andy讲解(2021) 原题链接

C++

```
#include <iostream>
using namespace std;

const int N = 10; //最大值
int n;             //读入的n
int path[N];       //记录路径每个位的值
bool used[N];      //记录第i个数是否被用过

void dfs(int u) {
    if (u == n) {
        //输出排列
        for (int i = 0; i < n; i++) cout << path[i]
<< " ";
        cout << endl;
        return;
    }
    //枚举数字1--n
    for (int i = 1; i <= n; i++) {
        if (!used[i]) {
            path[u] = i; //记录当前位path[u]的数字
            used[i] = true; //标记数字i为已经使用郭
            dfs(u + 1); //递归处理下一个位
            used[i] = false; //恢复现场
        }
    }
}

int main() {
    cin >> n;
    dfs(0);
    return 0;
}
```

Python

```
# AcWing 842
# Python solution
```

NQ131 DFS试炼之n皇后问题 AcWing 843

n-皇后问题是指将 n 个皇后放在 $n \times n$ 的国际象棋棋盘上，使得皇后不能相互攻击到，即任意两个皇后都不能处于同一行、同一列或同一斜线上。数据范围: $1 \leq n \leq 12$

输入 共一行，包含整数n。

输出 每个解决方案占n行，每行输出一个长度为n的字符串，用来表示完整的棋盘状态。其中“.”表示某一个位置的方格状态为空，“Q”表示某一个位置的方格上摆着皇后。每个方案输出完成后，输出一个空行。输出方案的顺序请根据样例，按照次序从小到大，从左到右输出。

来源 AcWing 843

输入

4

输出

```
.Q..
...Q
Q...
..Q.
..Q.
Q...
```

```
...Q
.Q..
```

提示 原题链接 Y总讲解

C++

```
#include <iostream>
using namespace std;

const int N = 20; //最大值
int n;             //输入的n
char g[N][N];      //棋盘
//三个指示器列、对角线、反对角线, 对角线以截距为编号
bool col[N], dg[N], udg[N];

void dfs(int u) {
    if (u == n) {
        //输出排列
        for (int i = 0; i < n; i++) puts(g[i]);
        puts("");
        return;
    }
    //枚举列0--n
    for (int i = 0; i < n; i++) {
        if (!col[i] && !dg[i + u] && !udg[i - u + n]) {
            g[u][i] = 'Q'; //当前位置设置为Q字符
            col[i] = dg[i + u] = udg[i - u + n] = true;
            //标记数字i为已经使用
            dfs(u + 1);
            //递归处理下一个位
            g[u][i] = '.';
            //恢复现场
            col[i] = dg[i + u] = udg[i - u + n] = false;
        }
    }
}

int main() {
    cin >> n;
    //初始化图
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) g[i][j] = '.';
    dfs(0);
    return 0;
}
```

Python

```
# AcWing 843
# Python solution
```

NQ132 BFS试炼之走迷宫 AcWing 844

给定一个 $n \times m$ 的二维整数数组，用来表示一个迷宫，数组中只包含0或1，其中0表示可以走的路，1表示不可通过的墙壁。最初，有一个人位于左上角(1, 1)处，已知该人每次可以向上、下、左、右任意一个方向移动一个位置。请问，该人从左上角移动至右下角(n, m)处，至少需要移动多少次。数据保证(1, 1)处和(n, m)处的数字为0，且一定至少存在一条通路。数据范围： $1 \leq n \leq 10$

输入

第一行包含两个整数n和m。接下来n行，每行包含m个整数（0或1），表示完整的二维数组迷宫。

输出

输出一个整数，表示从左上角移动至右下角的最少移动次数。

来源

AcWing 844

输入

```
5 5
0 1 0 0 0
0 1 0 1 0
0 0 0 0 0
0 1 1 1 0
0 0 0 1 0
```

输出

```
8
```

提示 [原题链接](#) [Y总讲解](#) [参考代码](#)

C++

```

#include <algorithm>
#include <iostream>
#include <queue>
#include <cstring>
using namespace std;

typedef pair<int, int> PII; // Y总风格pair<int, int>声明

const int N = 107;

int n, m;
int g[N][N], d[N][N];

//广搜
int bfs() {

    queue<PII> q; //队列

    memset(d, -1, sizeof d);
    d[0][0] = 0;
    q.push({0, 0});

    // 四个方向向量
    int dx[] = {-1, 0, 1, 0}, dy[] = {0, 1, 0, -1};

    while (q.size())
    { //队列为空的时候退出广搜
        auto t = q.front(); //取队头
        q.pop(); //弹出队头

        //搜索四个方向
        for (int i = 0; i < 4; i++)
        {
            int x = t.first + dx[i], y = t.second + dy[i]; //待搜索的新坐标点

            if (x < 0 || x >= n || y < 0 || y >= m || g[x][y] != 0 || d[x][y] != -1) continue;

            d[x][y] = d[t.first][t.second] + 1;

            q.push({x, y}); //入队
        }
    }

    return d[n-1][m-1];
}

int main() {

    cin >> n >> m;

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            cin >> g[i][j];

    cout << bfs() << endl;
}

```

Python

```

# AcWing 844
# Python solution

```

```
return 0;
}
```

NQ133 BFS试炼之八数码 AcWing 845

在一个3×3的网格中，1~8这8个数字和一个“x”恰好不重不漏地分布在这3×3的网格中。例如：1 2 3 x 4 6 7 5 8 在游戏过程中，可以把“x”与其上、下、左、右四个方向之一的数字交换（如果存在）。我们的目的是通过交换，使得网格变为如下排列（称为正确排列）：1 2 3 4 5 6 7 8 x 例如，示例中图形就可以通过让“x”先后与右、下、右三个方向的数字交换成功得到正确排列。交换过程如下：1 2 3 1 2 3 1 2 3 1 2 3 x 4 6 4 x 6 4 5 6 4 5 6 7 5 8 7 5 8 7 x 8 7 8 x 现在，给你一个初始网格，请你求出得到正确排列至少需要进行多少次交换。

输入 输入占一行，将3×3的初始网格描绘出来。例如，如果初始网格如下所示：1 2 3 x 4 6 7 5 8 则输入为：1 2 3 x 4 6 7 5 8

输出 输出占一行，包含一个整数，表示最少交换次数。如果不存在解决方案，则输出“-1”。

来源 AcWing 845

输入	输出
2 3 4 1 5 x 7 6 8	19

提示 [原题链接](#) [Y总讲解](#) [参考代码](#)

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>
#include <unordered_map>
#include <queue>

using namespace std;

int bfs(string start)
{
    string end = "12345678x";

    queue<string> q;
    unordered_map<string,int> d;//字符串映射到整数

    q.push(start);
    d[start] = 0;//map的用法

    //方向向量
    int dx[4] = {1,-1,0,0},dy[4]={0,0,1,-1};

    //广搜
    while(q.size())
    {
        auto t = q.front();
        q.pop();

        int distance = d[t];
        if (t==end) return distance;//找到目标

        //查询x在字符串中的下标
        int k = t.find('x');
        int x = k / 3, y = k % 3;//转换为宫格坐标

        for (int i = 0; i < 4; i ++ )
        {
            int a = x + dx[i], b = y + dy[i];

            if (a >= 0 && a < 3 && b >= 0 && b < 3)
            {
                swap(t[k], t[a*3+b]);//移动x的位置
                if (!d.count(t))
                {
                    d[t] = distance + 1;
                    q.push(t);//压入队列
                }
                swap(t[k],t[a*3+b]);//还原现场
            }
        }
    }
    return -1;
}

int main()
{
    string c, start;
    //去掉空格
    for (int i = 0; i < 9; i ++ )
    {
        cin>>c;
        start += c;
    }
}

```

Python

```

# AcWing 845
# Python solution

```

```
cout<<bfs(start)<<endl;  
}
```

NQ134 树的重心 AcWing 846

给定一颗树，树中包含 n 个结点（编号 $1\sim n$ ）和 $n-1$ 条无向边。请你找到树的重心，并输出将重心删除后，剩余各个连通块中点数的最大值。重心定义：重心是指树中的一个结点，如果将这个点删除后，剩余各个连通块中点数的最大值最小，那么这个节点被称为树的重心。数据范围： $1\leq n\leq 10^5$

输入

第一行包含整数 n ，表示树的结点数。接下来 $n-1$ 行，每行包含两个整数 a 和 b ，表示点 a 和点 b 之间存在一条边。

输出

输出一个整数 m ，表示重心的所有的子树中最大的子树的结点数目。

来源

AcWing 846

输入

```
9  
1 2  
1 7  
1 4  
2 8  
2 5  
4 3  
3 9  
4 6
```

输出

```
4
```

提示 [原题链接](#) [Y总讲解](#) [Y总代码](#)

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 100007, M = N*2; // 无向图

int n;
int h[N], e[M], ne[M], idx;
int ans = N;
bool st[N]; // 顶点n是否访问

void add(int a, int b) // 添加一条边a->b
{
    e[idx] = b, ne[idx] = h[a], h[a] = idx ++ ;
}

int dfs(int u)
{
    st[u] = true;

    int size = 0; // 去掉i节点后, i子树的最大子树节点数
    int sum = 0; // i子树的节点总数
    for (int i = h[u]; i != -1; i = ne[i])
    {
        int j = e[i];
        if (st[j]) continue;

        int s = dfs(j); // 返回j子树的大小
        size = max(size, s);
        sum += s; // 计算所有子树和
    }

    size = max(size, n - sum - 1); // i子树和与 非i子树部分的和, 取最大值
    ans = min(ans, size);

    return sum+1; // 加上i节点本身
}

int main()
{
    cin >> n;
    memset(h, -1, sizeof h);
    for (int i = 0; i < n-1; i ++ )
    {
        int a, b;
        cin >> a >> b;
        add(a, b), add(b, a); // 无向边
    }
    dfs(1);
    cout << ans << endl;
    return 0;
}

```

Python

```

# AcWing 846
# Python solution

```

给定一个 n 个点 m 条边的有向图，图中可能存在重边和自环。所有边的长度都是1，点的编号为 $1 \sim n$ 。请你求出1号点到 n 号点的最短距离，如果从1号点无法走到 n 号点，输出 -1 。数据范围： $1 \leq n, m \leq 10^5$

输入

第一行包含两个整数 n 和 m 。接下来 m 行，每行包含两个整数 a 和 b ，表示存在一条从 a 走到 b 的长度为1的边。

输出

输出一个整数，表示1号点到 n 号点的最短距离。

来源

AcWing 847

输入

4 5
1 2
2 3
3 4
1 3
1 4

输出

1

提示 [原题链接](#) [Y总讲解](#) [Y总代码](#)

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>
#include <queue>

using namespace std;

const int N = 100007; // 有向图

int n, m;
int h[N], e[N], ne[N], idx;
int d[N];

void add(int a, int b) // 添加一条边 a->b
{
    e[idx] = b, ne[idx] = h[a], h[a] = idx ++ ;
}

int bfs()
{
    memset(d, -1, sizeof d);
    queue<int> q;
    d[1] = 0; // 起始点
    q.push(1);

    while (q.size())
    {
        auto t = q.front();
        q.pop();

        for (int i = h[t]; i != -1; i = ne[i])
        {
            int j = e[i];
            if (d[j] == -1)
            {
                d[j] = d[t] + 1; // 层次加1
                q.push(j);
            }
        }
    }
    return d[n];
}

int main()
{
    cin >> n >> m;
    memset(h, -1, sizeof h);

    for (int i = 0; i < m; i++)
    {
        int a, b;
        cin >> a >> b;
        add(a, b);
    }

    cout << bfs() << endl;
    return 0;
}

```

Python

```

# AcWing 847
# Python solution

```

给定两个字符串a和b，定义 $a \times b$ 为它们的连接。例如，如果 $a = \text{"abc"}$ 而 $b = \text{"def"}$ ，则 $a \times b = \text{"abcdef"}$ 。如果将连接考虑成乘法，一个非负整数的乘方将用一种通常的方式定义： $a^0 = \text{" "}$ （空字符串）， $a^{(n+1)} = a \times (a^n)$ 。

输入 输入包含不超过 10 组测试样例，每组测试样例占一行。每组样例包含一个由小写字母构成的字符串s，s的长度不超过 100，且不包含空格。最后的测试样例后面将是一个点号作为一行。

输出 对于每一个s，需要输出最大的n，使得存在一个字符串a，让 $s = a^n$ 。

来源 AcWing 777

输入	输出
abcd	1
aaaa	4
ababab	3
.	

提示 注意输出格式。

C++

```
#include <iostream>

using namespace std;

int main()
{
    string str;

    while (cin >> str, str != ".")
    {
        int len = str.size();

        for (int n = len; n; n -- )
            if (len % n == 0)
            {
                int m = len / n;
                string s = str.substr(0, m);
                string r;
                for (int i = 0; i < n; i ++ ) r +=
s;

                if (r == str)
                {
                    cout << n << endl;
                    break;
                }
            }
    }

    return 0;
}
```

Python

```
# AcWing 777
# Python solution
```

有三个字符串S、S1、S2。其中，S长度不超过 300，S1和S2的长度不超过 10。现在，我们想要检测S1和S2是否同时在S中出现，且S1位于S2的左边，并在S中不交叉（即，S1的右边界点在S2的左边界点的左侧）。计算满足上述条件的最大跨距（即，最大间隔距离：最右边的S2的起始点与最左边的S1的终止点之间的字符数目）。如果没有满足条件的S1、S2存在，则输出-1。例如，S = abcd123dABdefghjsdef76ki，S1 = ab，S2 = ef。其中，S1在S中出现了若干次，S2也在S中出现了若干次，最大跨距为18。

输入

输入共一行，包含三个字符串S、S1、S2，字符串之间用逗号隔开。数据保证三个字符串中不含空格和逗号。

输出

输出一个整数，表示最大跨距。如果没有满足条件的S1和S2存在，则输出-1。

来源

AcWing 778

输入

abcd123dABdefghjsdef76ki,ab,ef

输出

18

提示 注意输出格式。

C++

```

#include <iostream>

using namespace std;

int main()
{
    string s, s1, s2;

    char c;
    while (cin >> c, c != ',') s += c;
    while (cin >> c, c != ',') s1 += c;
    while (cin >> c) s2 += c;

    if (s.size() < s1.size() || s.size() < s2.size()) puts("-1");
    else
    {
        int l = 0;
        while (l + s1.size() <= s.size())
        {
            int k = 0;
            while (k < s1.size())
            {
                if (s[l + k] != s1[k]) break;
                k ++ ;
            }
            if (k == s1.size()) break;
            l ++ ;
        }

        int r = s.size() - s2.size();
        while (r >= 0)
        {
            int k = 0;
            while (k < s2.size())
            {
                if (s[r + k] != s2[k]) break;
                k ++ ;
            }
            if (k == s2.size()) break;
            r -- ;
        }

        l += s1.size() - 1;

        if (l >= r) puts("-1");
        else printf("%d\n", r - l - 1);
    }

    return 0;
}

```

Python

```

# AcWing 778
# Python solution

```